

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_6e519d4b-9e6\agent-abf3a39c84cce5e8e.jsonl`
- SHA-256: `6886eff0c5b112431afbbb63f3fb83a6754d8193cb3d3f5bb64133809c859052`
- Source modified: `2026-06-22T14:48:13+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_6e519d4b-9e6`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T14:46:27.067Z)

You are designing ITEM 1 of the "local-app -> Cloud Run" feature: wire the API job path (/api/process -> backend/api/job_worker.py::_run_claimed_job) to DISPATCH to Cloud Run when deployment.compute_target=="cloud_run" (mirroring the worker CLI), poll the bucket, and INGEST the cloud result (rally segments) back into the API SQLite DB so the existing /api/query + /api/compile (local stitch) keep working. It MUST be default-OFF (byte-identical when compute_target != cloud_run) and FULLY testable offline with the injectable fake Cloud Run client (no real GPU). Produce a concrete, minimal-diff design.

Here are 5 verified maps of the current code:

```
=== worker-cli-dispatch ===
I have the full path mapped. Here is the exact dispatch flow.

---
```

How `backend.worker infer` dispatches a job to Cloud Run

Entry: `cmd\_infer` decides local-vs-cloud via the compute factory

`backend/worker/\_\_main\_\_.py:158` `cmd\_infer`. After config load + startup checks, the branch point is:

```
- `__main__.py:168` ? `compute = get_compute_backend(config)`
- `__main__.py:169-170` ? `if compute is not None: return _dispatch_remote(compute, args)`
```

The factory (`backend/compute/\_\_init\_\_.py:47-48`) returns non-`None` **only** when `config.deployment.compute\_target == "cloud\_run"`; `""` / `local\_gpu` / `cpu\_mock` ? `None` ? fall through to the inline pull?segment?push path (the rest of `cmd\_infer`). So cloud dispatch is gated purely on that config field. Cloud Run coordinates come from env, not config (`\_\_init\_\_.py:54-56`): `CLOUD\_RUN\_JOB` (default `rally-worker`), `CLOUD\_RUN\_REGION` (default `asia-south1`), `GOOGLE\_CLOUD\_PROJECT`.

Building the `ComputeJobSpec`

`\_dispatch\_remote` (`\_\_main\_\_.py:118-139`) constructs it directly from the CLI args:

```
...
__main__.py:128-131
spec = ComputeJobSpec(
    video_uri=args.video_uri, out_uri=args.out_uri,
    segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
)
result = compute.run_infer(spec) # __main__.py:133
```

...

``ComputeJobSpec`` (``backend/compute/base.py:20-30``) is a frozen dataclass: ``video_uri``, ``out_uri``, ``segmenter``, ``config_overrides: tuple[str,...]``. The frozen/tuple choice is deliberate (hashable spec).

Dispatch + poll ? ``CloudRunCompute.run_infer``

``backend/compute/cloud_run.py:103-132``. This is the heart of the mirror target:

```
1. **Derives a private done-marker URI** keyed by a per-call uuid token so concurrent jobs to
one bucket don't collide:
    - `cloud_run.py:104-106` ? `out_root = spec.out_uri.rstrip("/")`; `token = self._token or
uuid.uuid4().hex`; `done_uri = f"{out_root}/_dispatch/{token}.done"`
2. **Builds the container argv** ? literally the same `backend.worker infer` CLI, with `--done-uri` added:
    - `cloud_run.py:107-112`:
        ...
        argv = ["infer", "--video-uri", spec.video_uri, "--out-uri", spec.out_uri, "--done-uri",
done_uri]
        if spec.segmenter: argv += ["--segmenter", spec.segmenter]
        for kv in (*spec.config_overrides, *self._container_overrides): argv += ["--set", kv]
        ...
    - `self._container_overrides` defaults to `_DEFAULT_CONTAINER_OVERRIDES`
(`cloud_run.py:41-44`): `deployment.compute_target=local_gpu` and
`indexing.detector_impl=native`. These force the container to run **inline on the L4 GPU** and
disarm the dispatch guard so it never recursively re-dispatches.
3. **Fires the Cloud Run Job execution** (non-blocking):
    - `cloud_run.py:114-115` ? `execution = self._client.run_job(self._job_name, argv,
region=..., project=...)`. The real client (`GoogleCloudRunJobClient.run_job`,
`cloud_run.py:64-79`) lazy-imports `google.cloud.run_v2`, overrides the job's container `args`
with this `argv`, and starts it. It does **not** wait.
4. **Bucket-polls the done-marker** (NOT the Cloud Run API status ? D16):
    - `cloud_run.py:119-120` ? `marker = poll_until(lambda: self._read_marker(done_uri),
self._policy, ...)`. `_read_marker` (`cloud_run.py:135-140`) returns `None` (keep polling) until
the object `exists()` in storage, then reads it as JSON. `poll_until` runs under
`DEFAULT_POLICY`: `poll_every_n_sec=10`, `max_wait_sec=1800` total wall budget,
`op_timeout_sec=120` per attempt (`execution_policy.py:47-49`). Exhausting the budget raises
`PolicyTimeout`.
```

What is read/written in the bucket

```
**Container (the worker running inline on GPU) WRITES** ? `cmd_infer` inline path,
`__main__.py:238-261`:
- `__main__.py:239-242` ? serializes two files into `scratch/outputs/<video_id>/`:
    - **`intervals.csv` via `_write_intervals_csv` (`__main__.py:73-85`): header
`start,end,shot_count,ending_reason`, decimal-second `[start,end)` rallies ? the join-friendly
artifact matching the rally-annotator label schema.
    - **`report.json` via `_write_report_json` (`__main__.py:88-96`): `{video_id, status,
segment_count, segments:[{start,end}]}`.
- `__main__.py:244-249` ? pushes every file under `out_local` to
`{out_uri}/outputs/<video_id>/<fn>` via `storage.put`. Output path is keyed on the recomputed
MD5 `video_id` (`__main__.py:183`), so it's deterministic regardless of where the job ran.
- `__main__.py:259-260` ? because `--done-uri` was passed, writes the **completion marker** via
`_write_done_marker` (`__main__.py:142-155`): a tiny JSON `{video_id, returncode, segment_count,
status}` put to `done_uri`. **Written on SUCCESS only** (`__main__.py:258` comment) ? a
failed/hung container writes no marker, so the dispatcher's poll times out (failure-marker is a
noted follow-up).
```

```
**Dispatcher READS** ? `CloudRunCompute.run_infer`, `cloud_run.py:121-132`:
- From the marker: `video_id = marker.get("video_id","")` and `rc = int(marker.get("returncode",
1))`. A **missing returncode defaults to 1 (failure), never 0** (`cloud_run.py:123-124`); an
empty/unreadable marker (no `video_id`) is forced to failure (`cloud_run.py:125-128`).
- `cloud_run.py:129-130` ? derives `outputs_uri = f"{out_root}/outputs/{video_id}/` and reads
**`report.json`** from there into a dict: `report =`
```

```
self._read_json(f"{outputs_uri}report.json")`.
```

What the RESULT contains ? and the key mirroring caveat

```
`run_infer` returns a `ComputeResult` (`base.py:33-43`):  
...
```

```
cloud_run.py:132
```

```
return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id, report=report)  
...
```

Fields: `returncode`, `outputs\_uri`, `video\_id`, `report` (the parsed `report.json` dict).

****The rally segments are NOT returned in-memory across the cloud boundary.**** The dispatcher never sees the segment list as Python objects ? it only:

- gets back the ****summary `report.json`**** (which has `segment\_count` and a list of `{start, end}` only ? no `shot\_count`/`ending\_reason`), and
- knows `outputs\_uri`, the bucket prefix where the ****full `intervals.csv` + `report.json`**** durably live.

So to get the actual rally intervals, ****the caller must read**

``{out_uri}/outputs/{video_id}/intervals.csv`` (and/or ``report.json``) back from the bucket itself.****** ``_dispatch_remote`` (``__main__.py:137-139``) just logs ``video_id``, ``returncode``, ``outputs_uri`` and returns ``result.returncode`` as the worker exit code. ``_dispatch_remote`` also maps ``PolicyTimeout`` ? a clean ****rc 4**** (``__main__.py:134-136``), distinct from local rc 2 (validation/config) / rc 3 (segmenter failure).

Summary for the API mirror

To mirror this path, the API must:

1. Set ``deployment.compute_target=cloud_run`` (and have ``CLOUD_RUN_JOB``/``CLOUD_RUN_REGION``/``GOOGLE_CLOUD_PROJECT`` in env), then build a ``ComputeJobSpec(video_uri=<gs://in>, out_uri=<gs://out-root>, segmenter=..., config_overrides=...)`` and call ``get_compute_backend(config).run_infer(spec)`` ? or replicate ``run_infer``'s ``dispatch+poll`` directly.
2. Treat the call as ****async-but-blocking****: it dispatches a Cloud Run Job then bucket-polls a ``{out_uri}/_dispatch/<token>.done`` marker until done or ``PolicyTimeout`` (default 1800s wall budget). For a web API you will likely want to split dispatch from poll rather than block a request thread on ``poll_until``.
3. On completion, take ``result.video_id`` + ``result.outputs_uri`` and ****fetch `intervals.csv` from the bucket**** to get the full rally segments (start/end/shot_count/ending_reason) ? ``result.report`` only carries the coarse ``{start,end}`` summary, and nothing returns segments in-memory.

Relevant files (absolute):

- ``C:/Users/avidu/Projects/bhi-cloudrun/backend/worker/__main__.py`` (``cmd_infer`` :158, ``_dispatch_remote`` :118, writers :73/:88, done-marker :142, marker write :259)
- ``C:/Users/avidu/Projects/bhi-cloudrun/backend/compute/__init__.py`` (factory gate :47)
- ``C:/Users/avidu/Projects/bhi-cloudrun/backend/compute/base.py`` (``ComputeJobSpec`` :20, ``ComputeResult`` :33)
- ``C:/Users/avidu/Projects/bhi-cloudrun/backend/compute/cloud_run.py`` (``run_infer`` :103, argv build :107, poll :119, result :132, marker read :135)
- ``C:/Users/avidu/Projects/bhi-cloudrun/backend/infrastructure/execution_policy.py`` (``poll_until``, ``PolicyTimeout`` :39, ``ExecutionPolicy`` defaults :44-53)

=== api-inprocess-path ===

I have the complete map. Here is the exact in-process flow.

How the API job flow runs a job IN-PROCESS today

All file:line refs are in ``bhi-cloudrun`` (HEAD ``df3b225``).

0. Submit ? `POST /api/process` creates a `queued` row, kicks the worker

```
`backend/main.py:559-608` (`process_video`):  
- Fast-fail validation (path, file existence, unknown segmenter) ? immediate 4xx.  
- `edge_auth.resolve_tenant(...)` ? `owner_id` (default-OFF ? `None`), `main.py:572`.  
- Generates `job_id = uuid.uuid4().hex` (`main.py:599`), inserts a row via  
`db_instance.create_job(...)` (`main.py:600-603`). Returns **202** `{"job_id",  
"status":"queued", "poll":"/api/jobs/{job_id}"}` (`main.py:605-608`).  
- Kicks the drain: `_get_executor().submit(_drain_due_jobs)` (`main.py:604`). The executor is  
`ThreadPoolExecutor(max_workers=1)` (`backend/api/job_worker.py:33-38`) ? single-worker by  
design.
```

```
`create_job` (`backend/infrastructure/database.py:623-640`) inserts into the `jobs` table with  
`status='queued'`; schema is `_migrate_to_v2` at `database.py:249-267` (cols: `id`, `video_path`,  
`video_id`, `segmenter`, `player_pool`, `max_frames`, `status`, `progress`, `error`, `result`, `created_at`,  
`started_at`, `finished_at`), plus v3 `policy`/`next_poll_at` (`database.py:276-277`) and v6 cost  
cols. `owner_id`/`locale` are later-migration cols.
```

1. Drain loop ? claim then run

```
`_drain_due_jobs` (`job_worker.py:132-153`): loops `db_instance.claim_due_job()` ?  
`_run_claimed_job(job)` until `claim_due_job` returns `None`, then  
`_schedule_next_drain_if_waiting()`.
```

```
`claim_due_job` (`database.py:831-854`) is the **queued ? running** transition: atomic compare-  
and-set ? SELECTs the earliest runnable job (`status='queued'` OR a `waiting` job whose  
`next_poll_at` is due, `database.py:840-842`), then `UPDATE jobs SET status='running',  
started_at=CURRENT_TIMESTAMP WHERE id=? AND status=?` (`database.py:845-847`). If `cur.rowcount  
!= 1` it lost the race ? returns `None` (`database.py:849-850`). On success returns the full job  
dict via `get_job` (`database.py:851`). (`mark_job_running` at `database.py:652-657` exists but  
the actual claim flips status inline here.)
```

2. Run one claimed job ? `\_run\_claimed\_job`

```
`job_worker.py:72-98`. Resolves everything through `backend.main`:  
- `req = _main._job_to_request(job)` ? rebuilds a `ProcessRequest` from the row  
(`job_worker.py:59-69`); `ProcessRequest` shape at `main.py:259-266` (`video_path`, `player_pool`,  
`max_frames`, `segmenter`, `locale`).  
- `result = _main._execute_processing(req, progress=lambda stage:  
db_instance.set_job_progress(job_id, stage))` (`job_worker.py:86-87`). The progress callback  
writes the coarse stage string to `jobs.progress` (`set_job_progress`, `database.py:659-663`).  
- On success: if `result["video_id"]`, `db_instance.set_job_video_id(job_id, ...)`  
(`job_worker.py:88-89`, `database.py:665-669`); then `db_instance.mark_job_done(job_id, result)`  
(`job_worker.py:90`) ? the **running ? done** transition; then `_maybe_record_job_cost` (M8,  
default-OFF no-op, `job_worker.py:91`, `:101-129`).  
- On `JobWaiting` (`job_worker.py:92-95`): `set_job_waiting(...)` parks it (**running ?  
waiting**) ? not used by the production path today (`job_worker.py:41-56`).  
- On any other `Exception` (`job_worker.py:96-98`): `db_instance.mark_job_failed(job_id,  
"{type}: {e}")` ? the **running ? failed** transition.
```

3. The pipeline ? `\_execute\_processing`

```
`backend/main.py:439-556`. This is the former synchronous `/api/process` body. Sequence:  
1. `notify("hashing")` ? `storage.acquire_local_input(req.video_path, ...)` resolves URI/path to  
a local file (`main.py:458`); `video_id = compute_video_id(local_video)` (MD5, `main.py:459`);  
`was_reingest = db_instance.get_video(video_id) is not None` (`main.py:460`).  
2. `notify("validating")` ? `registry.run_all_with_context(...)` (`main.py:465`). If any  
validator fails (`main.py:475-485`): `db_instance.add_video(video_id, req.video_path, "failed",  
[...])` (UPSERT, no cascade ? `database.py:405-422`), returns  
`{"video_id", "status": "failed", "message", "results"}`.  
3. Else: `db_instance.add_video(video_id, req.video_path, "processed", [...])` (`main.py:489`).  
Resolve segmenter: `seg_name = req.segmenter or default_seg` (`main.py:469-470`); `segmenter_cls`  
= `segmenter_registry.get(seg_name)` (`main.py:491`).  
4. **Destroy-after-confirm**: capture `play_wm`, `cand_wm` =  
`db_instance.segment_watermarks(video_id)` (`main.py:510`, `database.py:424-436`). Build  
segmenter: `segmenter = segmenter_cls(db_instance, global_config)` (`main.py:511`).  
5. **Run the core**: `result = segmenter.process_video(video_id, local_video, req.player_pool,  
req.max_frames)` (`main.py:513`). On raise ? `prune_segments(... play_after=play_wm,
```

cand_after=cand_wm)` drops new partial rows and re-raises (`main.py:514-519`). On `Failure` ? same prune + `status:"failed"` response (`main.py:522-532`).

6. `segments = result.value` (`main.py:534`); `notify("finalizing")`;

`\_finalize\_reingest(video\_id, segments, play\_wm, cand\_wm)` (`main.py:536`, `:283-293`) ? on a non-empty run drops the PRIOR rows (`prune\_segments play\_upto=play\_wm`); on empty drops the NEW rows (keeps prior good data).

7. Returns the success dict (`main.py:538-544`): `{"video\_id", "status":"success", "message", "results":[validators], "play\_segments": segments}`.

8. `finally:` always emits the per-video observability report and grafts `response["reports"] = paths` (`main.py:546-556`).

****How segments actually land in SQLite:**** `\_execute\_processing` does ****not**** insert segments itself. The segmenter (`segmenter.process\_video`, ABC at `backend/pipeline/segmenters/base.py:23-33`) writes them ****as a side effect**** via its injected `self.db`, calling `db\_instance.add\_play\_segment(video\_id, start, end, server, receiver, metadata)` (`database.py:459-471` ? `INSERT INTO play\_segments (...) VALUES (...)`, returns `lastrowid`) and optionally `add\_candidate\_segment` (`database.py:480-501`) / `add\_event` (`database.py:473-478`). The returned `result.value` list is the same segments, used for the count + watermark finalize. So persistence is: ****segmenter ? `self.db.add\_play\_segment` ? `play\_segments` table****, with `\_execute\_processing` only managing the watermark/prune lifecycle around it.

result dict shape stored on the job

`mark\_job\_done(job\_id, result)` (`database.py:671-679`) does `UPDATE jobs SET status='done', progress='finished', result=<json.dumps(result)>, finished\_at=CURRENT\_TIMESTAMP`. The stored `result` is exactly the `\_execute\_processing` return dict:

```
...
{"video_id", "status": "success"|"failed", "message", "results": [...validators...],
 "play_segments": [...] (success/segmenter-fail branches), "reports": {...} (grafted in finally)}
...
```

Note `result["status"]` is the ****pipeline verdict**** (`success`/`failed`), distinct from the job's ****execution**** `status` column (`done`).

status transitions (jobs.status)

`queued` (create_job, `database.py:635`) ? `running` (claim_due_job CAS, `database.py:846`) ? terminal `done` (mark_job_done, `database.py:675`) ****or**** `failed` (mark_job_failed on worker crash, `database.py:684`) ****or**** parked `waiting` (set_job_waiting, `database.py:802`, unused in prod) which re-enters `claim\_due\_job` when `next\_poll\_at` is due. Crash recovery: `fail\_stale\_jobs` (`database.py:856-870`) sweeps any `queued`/`running` rows to `failed` at startup (the in-process executor died with the process).

GET /api/query reads segments afterward

`POST /api/query` (`main.py:694-703`, body `QueryRequest` at `main.py:268-273`) calls `db\_instance.query\_play\_segments(req.video\_id, filters)` and returns `{"play\_segments": segments}`. `query\_play\_segments` (`database.py:564-612`) does `SELECT r.\* FROM play\_segments r WHERE r.video\_id = ?` with optional `duration\_min` / `server` / `receiver` / `event\_type` (EXISTS-subquery on `events`) filters, deserializes `metadata` JSON, and attaches each segment's `events` (`database.py:597-610`). It reads ****only**** `play\_segments`/`events` keyed by `video\_id` ? it has no knowledge of jobs. `/api/compile` (`main.py:705-717`) reads the same way and filters by the client-supplied `segment\_ids`.

Where the cloud-dispatch branch must slot in

The cloud-dispatch interface ****already exists but is NOT wired****: `backend/compute/` has `ComputeBackend` ABC + `ComputeJobSpec`/`ComputeResult` (`backend/compute/base.py`, full file above) and `cloud\_run.py` (155 lines). ****`backend/main.py` never imports `backend.compute`**** (grep confirmed ? only `compute\_video\_id` and `compute\_target` from `/api/capabilities` appear). So today the dispatch decision is implicit: `\_execute\_processing` always runs the segmenter in-process on the worker thread.

The natural insertion point is ****`\_execute\_processing` at `main.py:511-513`**** ? right where it

builds `segmenter = segmenter_cls(db_instance, global_config)` and calls `segmenter.process_video(...)`. Per `base.py:8-10`, the seam is gated on `deployment.compute_target == "cloud_run"`: a factory returns a `ComputeBackend` (cloud) or `None` (run inline / byte-identical today). The cloud branch would replace the local `process_video` call with `backend.run_infer(ComputeJobSpec(video_uri=..., out_uri=..., segmenter=seg_name, ...))`, then ingest the remote `ComputeResult.outputs_uri` artifacts back into `play_segments` so that the downstream contract ? the same `result` dict stored by `mark_job_done`, and `query_play_segments` reading `play_segments` by `video_id` ? stays identical. The watermark/`_finalize_reingest` lifecycle (`main.py:510, 536`) and the `result["video_id"]`/`play_segments` shape must be preserved for `/api/query` and `/api/compile` to keep working unchanged.

Relevant files (absolute):

- `C:\Users\avidu\Projects\bhi-cloudrun\backend\api\job_worker.py` ? worker drain/claim/run loop.
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\main.py` ? `_execute_processing` (439-556), `/api/process` (559-608), `/api/query` (694-703), `/api/compile` (705-717), `ProcessRequest`/`QueryRequest` (259-273).
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\infrastructure\database.py` ? jobs schema (249-279), `add_video`/`segment_watermarks`/`prune_segments`/`add_play_segment` (405-471), `query_play_segments` (564-612), job store (623-870).
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\base.py` ? `process_video` ABC contract (23-33).
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\base.py` + `cloud_run.py` ? the unwired `ComputeBackend` dispatch seam the cloud branch plugs into.

=== compute-contract ===

All three files read fully. Here is the documented contract.

`backend/compute/` contract (COMPUTE_DECOUPLED_SERVING M6)

All file refs are absolute paths under `C:/Users/avidu/Projects/bhi-cloudrun/backend/compute/`.

`ComputeJobSpec` ? `base.py:20-30`

Frozen dataclass, one inference job, profile-agnostic.

- `video_uri`: str (`base.py:26`) ? input; `gs://?` for cloud, a path / `local://` for local.
- `out_uri`: str (`base.py:27`) ? output ROOT; the `gs://?` bucket for cloud.
- `segmenter`: Optional[str] = None (`base.py:28`).
- `config_overrides`: tuple[str, ...] = () (`base.py:30`) ? raw worker `--set k=v` overrides; a tuple (not list/dict) so the spec stays frozen/hashable.

`video_id` is NOT a field ? it's the MD5 the worker re-computes from the same input bytes, so output always lands at `<out_uri>/outputs/<video_id>/` deterministically (`base.py:22-24`).

`ComputeResult` ? `base.py:33-43`

Frozen dataclass, terminal state of a dispatched job.

- `returncode`: int (`base.py:40`) ? mirrors the worker exit code: `0` ok / `2` validation-or-config / `3` segmenter-failure, `**plus 4` = remote dispatch timeout or empty/unreadable completion marker (cloud path only) (`base.py:36-38`).
- `outputs_uri`: str = "" (`base.py:41`) ? where `intervals.csv` / `report.json` landed.
- `video_id`: str = "" (`base.py:42`) ? known once the done-marker is read.
- `report`: dict = field(default_factory=dict) (`base.py:43`).

`ComputeBackend.run_infer()` ? `base.py:46-53`

Note: the ABC method is `run_infer(self, spec: ComputeJobSpec) -> ComputeResult` (`base.py:51-53`), NOT `dispatch()`. It "dispatches ONE inference job and returns only once its outputs are durably available (bucket-poll for cloud), so a caller treats local and cloud execution identically" (`base.py:47-49`).

The factory seam ? `get_compute_backend()` ? `__init__.py:32-61`

Signature: `get_compute_backend(config, *, run_client=None, policy=DEFAULT_POLICY, token=None) -> Optional[ComputeBackend]` (`__init__.py:32-38`).

- Default-OFF: returns `None` unless `config.deployment.compute_target == "cloud_run"` (`__init__.py:47-49`). `None` means "run the worker's in-process path, byte-identical."

- Only `"cloud_run"` returns a backend (a `CloudRunCompute`). Coordinates come from control-plane env, no secrets in config: `CLOUD_RUN_JOB` (default `"rally-worker"`), `CLOUD_RUN_REGION` (default `"asia-south1"`), `GOOGLE_CLOUD_PROJECT` (`__init__.py:54-56`).
- **Client injection point:** `client = run_client` if `run_client` is not None else `GoogleCloudRunJobClient()` (`__init__.py:57`). So a test passes `run_client=<fake>` here; production defaults to the real client. The SDK is lazy-imported only on the cloud branch (`__init__.py:52`).

Done-marker / bucket-poll mechanism ? `cloud_run.py`

The flow (documented `cloud_run.py:4-12`):

1. **Dispatch** (`cloud_run.py:103-117`): builds `argv` then calls `self._client.run_job(...)` which does NOT block. The `argv`:

```

infer --video-uri <spec.video_uri> --out-uri <spec.out_uri> --done-uri <done_uri>

```


(`cloud_run.py:107-108`), then `--segmenter` if set (`cloud_run.py:109-110`), then one `--set kv` per entry in `(*spec.config_overrides, *self._container_overrides)` (`cloud_run.py:111-112`).
- `done_uri = f"{out_root}/{dispatch}/{token}.done"` (`cloud_run.py:106`), where `out_root = spec.out_uri.rstrip("/")` (`cloud_run.py:104`) and `token = self._token or uuid.uuid4().hex` (`cloud_run.py:105`). The token keys the marker so concurrent jobs to one bucket don't collide; `None` ? fresh `uuid4` per call (prod), injected ? deterministic (tests) (`cloud_run.py:96-98`).
2. **Bucket-poll** (`cloud_run.py:119-120`): `poll_until(lambda: self._read_marker(done_uri), self._policy, label="cloud-run-infer", logger=LOG)`. `poll_until` is imported from `backend.infrastructure.execution_policy` (`cloud_run.py:33`). The completion signal is a storage-backed marker object the worker writes, NOT the Cloud Run API status (D16, `cloud_run.py:6-10`); scale-to-zero cold-start is absorbed by the async poll (`cloud_run.py:10`).
- `_read_marker(done_uri)` (`cloud_run.py:135-140`): parses the URI, returns `None` if `get_storage(ref.backend, config=self._storage_config).exists(ref)` is false ? `poll_until` keeps waiting; else returns `self._read_json(done_uri)` or `{}` (an empty dict still terminates the poll ? `"present == done"`).
- `_read_json` (`cloud_run.py:142-155`): downloads to a real local temp dst FIRST (`tempfile.mkdtemp`), because cloud backends (gcs/s3) REQUIRE a dst ? a `None` dst raises `ValueError`, unlike local's identity passthrough; without an explicit dst this read would silently return `{}` for every `gs://|s3://` object (`cloud_run.py:143-145`). Returns `{}` on any exception.
3. **Read** (`cloud_run.py:121-132`): from the marker, `video_id = str(marker.get("video_id", ""))` (`cloud_run.py:121`) and `rc = int(marker.get("returncode", 1))` ? **a missing returncode defaults to FAILURE (1), never success** (`cloud_run.py:122-124`). If `video_id` is empty ? warn + `rc = rc or 1` (`cloud_run.py:125-128`). `outputs_uri = f"{out_root}/outputs/{video_id}/"` if a `video_id` exists else `""` (`cloud_run.py:129`); then `report = self._read_json(f"{outputs_uri}report.json")` (`cloud_run.py:130`). Returns `ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id, report=report)` (`cloud_run.py:132`).

Real vs injectable-fake Cloud Run client ? this is how item 1 is tested with NO real GPU

CloudRunJobClient ABC (`cloud_run.py:47-57`): one abstract method

```

run_job(self, job_name: str, argv: List[str], *, region: str, project: Optional[str]=None) -> str

```

Starts a Cloud Run Job execution running `python -m backend.worker <argv>`, returns an execution id/name for logs, and "Does NOT block on completion (the caller bucket-polls)" (`cloud_run.py:54-57`).

GoogleCloudRunJobClient (real) ? `cloud_run.py:60-79`: lazy-imports `google.cloud.run_v2` (`cloud_run.py:67`); raises `RuntimeError` if the SDK is absent (`cloud_run.py:68-72`, "intentionally NOT a CI/test dependency"). Builds `JobsClient()` ? `job_path(project, region, job_name)` ? a `RunJobRequest.Overrides` with a single `ContainerOverride(args=argv)` ? `client.run_job(...)`. Every real-SDK line is marked `# pragma: no cover` ? "CI uses a fake, so

this code path is never exercised in tests" (`cloud\_run.py:61-62`). Owner-verified on the M7 real run.

****How the fake is injected (no GCP / no GPU):**** `CloudRunCompute.\_\_init\_\_` takes `run\_client: CloudRunJobClient` as a required keyword-only arg (`cloud\_run.py:85-90`) and stores it as `self.\_client` (`cloud\_run.py:90`). A test subclasses `CloudRunJobClient` with a fake whose `run\_job` (a) records the argv and (b) ? in the same call or via a paired storage stub ? writes the done-marker JSON (`{"video\_id": ..., "returncode": ...}`) to `done\_uri` in the configured storage so `\_read\_marker` finds it and `poll\_until` terminates. The fake reaches `CloudRunCompute` either directly via `CloudRunCompute(run\_client=fake, ?)` or through the factory via `get\_compute\_backend(config, run\_client=fake)` (`\_\_init\_\_.py:35,57`). The `token` kwarg makes `done\_uri` deterministic for assertions (`cloud\_run.py:96-98,105-106`). Because the storage layer (`backend.storage`) already has a `local`/in-memory backend, the entire dispatch?marker?poll?read?`ComputeResult` cycle runs offline with no `google-cloud-run`, no Cloud Run, and no GPU.

What `CloudRunCompute` assumes about the deployed Cloud Run Job image

- ****The container runs the SAME worker entrypoint**** `python -m backend.worker` and the client only overrides its `args` per execution (`cloud\_run.py:5,48-51,54-57,76-78`). The image's baked entrypoint must therefore be `python -m backend.worker` (the override supplies only argv, e.g. `["infer", "--video-uri", ?]`).
- ****The worker CLI must accept these flags:**** `infer`, `--video-uri`, `--out-uri`, `--done-uri`, `--segmenter` (optional), and repeatable `--set k=v` (`cloud\_run.py:107-112`).
- ****The worker must write the done-marker**** to `--done-uri` as a JSON object carrying `video\_id` and `returncode` (both always written ? `cloud\_run.py:38,121-124`), and write `report.json` (and `intervals.csv`) under `/outputs/<video\_id>/` (`cloud\_run.py:11-12,129-130`; `base.py:22-24,38`).
- ****Forced INLINE execution to prevent recursive re-dispatch**** via the default container overrides `\_DEFAULT\_CONTAINER\_OVERRIDES` (`cloud\_run.py:41-44`), passed as `--set`:
 - `deployment.compute\_target=local\_gpu` ? disarms the `cmd\_infer` dispatch guard so the container computes instead of re-dispatching.
 - `indexing.detector\_impl=native` ? the L4 WASB runner.These are appended after `spec.config\_overrides` (`cloud\_run.py:111`) and are overridable via the `container\_overrides` ctor kwarg (`cloud\_run.py:89,99-101`).
- ****Env-supplied coordinates**** (from the factory, not the image): `CLOUD\_RUN\_JOB`, `CLOUD\_RUN\_REGION`, `GOOGLE\_CLOUD\_PROJECT` (`\_\_init\_\_.py:54-56`) ? these name the deployed Job, its region, and project.
- ****Storage assumption:**** the worker writes to, and the dispatcher reads from, the same storage bucket via `backend.storage` using `self.\_storage\_config` (passed from `config.storage` via `\_storage\_dict`, `\_\_init\_\_.py:23-29,60`); the marker and outputs are `gs://`/`s3://` objects, which is why `\_read\_json` forces a local dst (`cloud\_run.py:143-149`).

One observation worth flagging: the module docstring (`cloud\_run.py:1`) and `base.py:47` say the dispatcher "bucket-polls", but `ComputeBackend`'s concrete method is `run\_infer`, while `\_\_init\_\_.py`'s docstring at line 45 references "the real `google-cloud-run` client". There is no method literally named `dispatch()` anywhere ? the dispatch step is the internal `self.\_client.run\_job(...)` call inside `run\_infer` (`cloud\_run.py:114`). If your task spec refers to a `dispatch()` method, it does not exist by that name; the public entrypoint is `ComputeBackend.run\_infer(spec)`.

=== config-storage-resultformat ===

I now have the complete picture. Let me verify the exact paths/marker key format by confirming the `\_dispatch/<token>.done` and reviewing `parse\_uri`/`ref.name` quickly for the marker filename used in the poll.

The map is complete. Here is the full picture.

How `compute\_target=cloud\_run` + `storage=gcs` get set and flow to `get\_compute\_backend`

****1. The `cloud-serving` profile preset bundles all four knobs**** ? `presets.py:51-56`:

```
python
"cloud-serving": {
```



```

    "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
    "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
    "storage": {"backend": "gcs"},
},
...

```

The 1:1 profile?target map is `presets.py:32-36` (`COMPUTE\_TARGET\_FOR\_PROFILE["cloud-serving"] = "cloud\_run"`). Canonical enum tuples are `presets.py:27-28`.

****2. The loader applies the preset only when explicitly selected**** (`config.json` deployment.profile` or `DEPLOYMENT_PROFILE` env ? `loader.py:53-77` \_resolve\_explicit\_profile`). On a hit it does a 3-way deep merge `defaults < preset < config.json` (`loader.py:166-167`) and re-stamps the profile + back-fills the canonical `compute\_target` if config blanked it (`loader.py:170-187`). Auto-detect (`presets.py:109-132` suggest_profile`, keyed off `K_SERVICE`) only ever *suggests*, never applies (`loader.py:191-197`, D15). The typed mirror that pins parity is `DeploymentConfig` ? `models.py:362,365` (`compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"]`). Storage backend literal is `models.py:318`.

****3. `cmd\_infer` consults the compute seam first**** ? `worker/\_\_main\_\_.py:168-170`:

```

`python
compute = get_compute_backend(config)
if compute is not None:
    return _dispatch_remote(compute, args)
...

```

`get\_compute\_backend` returns non-None ****only**** for `compute\_target == "cloud\_run"` (`compute/\_\_init\_\_.py:47-49`); every other target returns `None` ? the inline in-process path runs (default-OFF seam). For cloud it lazy-imports and builds `CloudRunCompute`, pulling Cloud Run coordinates from env (`CLOUD\_RUN\_JOB`/`CLOUD\_RUN\_REGION`/`GOOGLE\_CLOUD\_PROJECT`), never config ? `compute/\_\_init\_\_.py:52-61`.

****4. Dispatch + completion flow**** (`compute/cloud\_run.py`): `run\_infer` (`:103-132`) starts a Cloud Run ****Job**** execution running the *same* `backend.worker\_infer` argv, forced inline via `\_DEFAULT\_CONTAINER\_OVERRIDES = ("deployment.compute\_target=local\_gpu", "indexing.detector\_impl=native")` (`:41-44`) so the container never re-dispatches, then ****bucket-polls**** the done-marker (`:119`, `poll\_until` + `read\_marker` at `:135-140`). The container worker writes the marker at the end of `cmd\_infer` only when `--done-uri` is passed (`worker/\_\_main\_\_.py:259-260`).

****Storage is fully credential-free + lazy****: `gcs` backend (`storage/gcs.py:17-25`) uses google ADC (`GOOGLE\_APPLICATION\_CREDENTIALS`/workload identity), imported lazily only when a `gs://` URI is touched. `acquire\_local\_input` (`storage/\_\_init\_\_.py:59-74`) fetches a remote input to a scratch dir (local/bare = identity passthrough); the worker uses the lower-level `storage.fetch`/`storage.put` directly (`worker/\_\_main\_\_.py:179, 248`).

ON-BUCKET OUTPUT FORMAT a cloud `worker infer` writes (what the API ingests back)

Written to scratch `outputs/<video\_id>/` (`worker/\_\_main\_\_.py:239-242`) then pushed to ****`<out\_uri>/outputs/<video\_id>/<fn>`**** for each file (`worker/\_\_main\_\_.py:244-249`). `video\_id` = whole-file MD5 of the *pulled bytes* (`worker/\_\_main\_\_.py:182-183`), so the path is deterministic regardless of where the job ran (`compute/base.py:22-24`).

File 1 ? `outputs/<video\_id>/intervals.csv`

Writer `\_write\_intervals\_csv` ? `worker/\_\_main\_\_.py:73-85`. Header + rows (`:78-84`):

```

| column | source | format |
|---|---|---|
| `start` | `s["start_time"]` | decimal seconds, `%.3f` |
| `end` | `s["end_time"]` | decimal seconds, `%.3f` |
| `shot_count` | `s["shot_count"]` | int or `""` |
| `ending_reason` | `s["ending_reason"]`, fallback `s["shot_count_confidence"]` | string or `""`
|

```

Semantics: decimal-second `[start, end)` rallies; docstring (`:74-75`) notes it is "the same

schema as the rally-annotator labels" ? the join-friendly artifact.

File 2 ? `outputs/<video\_id>/report.json`

Writer `\_write\_report\_json` ? `worker/\_\_\_main\_\_.py:88-96`. Keys:

```
```json
{
 "video_id": "<md5>",
 "status": "success" | "failed",
 "segment_count": <int>,
 "segments": [{ "start": <float>, "end": <float> }, ...]
}
...

```

Note `report.json` segments carry **only** `start`/`end` (floats) ? `shot\_count`/`ending\_reason` live **only** in `intervals.csv`. `status` is `success` when segmentation ran (`worker/\_\_\_main\_\_.py:214`), else `failed` (default `:195`; set on validation/segmenter failure paths). The cloud dispatcher reads this report back into `ComputeResult.report` (`compute/cloud\_run.py:129-132`).

### File 3 ? the done/success marker: `/\_dispatch/<token>.done`

This is the success signal, written **separately** from the `outputs/` prefix. The dispatcher chooses the URI as `f"{out\_root}/\_dispatch/{token}.done"` (`compute/cloud\_run.py:104-107`, `token` = injected or a fresh `uuid4().hex` `:105`) and passes it as `--done-uri`. The worker writes it via `\_write\_done\_marker` ? `worker/\_\_\_main\_\_.py:142-155` ? **on success only** (`:259-260`). Marker JSON (`:147-148`):

```
```json
{"video_id": "<md5>", "returncode": <int>, "segment_count": <int>, "status": "<status>"}
...

```

Caveats the API must handle:

- **No failure marker.** A failed/hung container writes no marker; the dispatcher's bucket-poll exhausts its budget ? `PolicyTimeout` ? clean **rc 4** (`worker/\_\_\_main\_\_.py:132-139`, `compute/base.py:38-43`). On the inline-success path the marker is always written with `returncode=0` hardcoded (`worker/\_\_\_main\_\_.py:260`).
- **Marker read defensiveness:** a missing/garbled `returncode` defaults to **failure (1)**, and a present-but-empty marker (no `video\_id`) is treated as failure (`compute/cloud\_run.py:121-128`).

Ingestion contract summary (what the API needs)

1. Poll/read `/\_dispatch/<token>.done` ? get `video\_id`, `returncode`, `status`, `segment\_count`.
2. On rc 0, fetch `/outputs/<video\_id>/report.json` (segment list + status) and/or `/outputs/<video\_id>/intervals.csv` (the richer per-rally rows incl. `shot\_count`/`ending\_reason`).
3. Key everything by `video\_id` (whole-file MD5; the labels?video?DB join key, `compute\_video\_id`).

Note: the worker writes **only** `intervals.csv` + `report.json` to the bucket (it lists scratch `outputs/<video\_id>/` and pushes each, `worker/\_\_\_main\_\_.py:244-249`). It does **not** push the SQLite DB, the highlight reel, or per-clip media ? the worker's DB (`worker/\_\_\_main\_\_.py:189`) and `emit\_indexer\_report` (`:231-236`) stay in the ephemeral scratch dir and are not uploaded. If the API needs the reel/DB, that is a gap (no compiled `highlights.mp4` is produced or pushed by `cmd\_infer`).

Relevant files (absolute):

- `C:\Users\avidu\Projects\bhi-cloudrun\backend\config\presets.py`
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\config\models.py`
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\config\loader.py`
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\\_\_init\_\_.py` (the `get\_compute\_backend` seam)
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\base.py`
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\cloud\_run.py`
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\\_\_\_main\_\_.py` (`cmd\_infer` + the three writers `\_write\_intervals\_csv` / `\_write\_report\_json` / `\_write\_done\_marker`)

```
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\storage\__init__.py`,  
`C:\Users\avidu\Projects\bhi-cloudrun\backend\storage\gcs.py`
```

=== tests-deploy ===

Confirmed. `\_execute\_processing` runs the pipeline in-process and never calls
`get\_compute\_backend` / never reads a cloud `outputs\_uri` back into the DB. I have everything
needed.

Summary

****Repo state:**** `bhi-cloudrun` is in detached HEAD (`## HEAD (no branch)`), so I could not `git
pull --ff-only` (no branch to track). Read-only on the local checkout; may lag remote.

1. Cloud-serving TESTS ? what's already covered (all \$0, GPU-free, cloud-free)

****File:**** `C:\Users\avidu\Projects\bhi-cloudrun\tests\test\_compute\_backend.py` (290 lines, the
single dedicated suite; `test\_billing.py:88,93` only asserts `compute\_backend="cloud\_run"` flows
through the cost ledger).

The mocked dispatch flow proven (`CloudRunCompute` in `backend/compute/cloud\_run.py`):

- ****Factory default-OFF**** (`test\_factory\_is\_none\_for\_local\_targets:66`,
`test\_factory\_returns\_cloud\_run\_for\_cloud\_target:73`): `get\_compute\_backend` returns `None` for
`""`/`local\_gpu`/`cpu\_mock`, a `CloudRunCompute` only for `compute\_target=cloud\_run`.
- ****Dispatch ? bucket-poll ? read**** (`test\_cloud\_run\_dispatch\_polls\_and\_returns\_result:80`): one
`run\_job` call to the right region/project; `ComputeResult` carries `returncode`, `video\_id`,
`outputs\_uri = {out}/outputs/{video\_id}/`, and `report.segment\_count`.
- ****argv construction**** (`test\_dispatch\_argv\_forces\_inline\_and\_carries\_overrides:97`):
dispatched container is forced `INLINE` ? appends `deployment.compute\_target=local\_gpu` +
`indexing.detector\_impl=native` AFTER user overrides (so user wins, and it never re-dispatches);
`--done-uri`/`--video-uri`/`--out-uri` always present.
- ****Token uniqueness**** (`test\_unique\_token\_per\_call\_when\_not\_injected:119`): fresh uuid per
dispatch so concurrent done-markers don't collide.
- ****Cloud-backend dst guard**** (`test\_read\_json\_passes\_dst\_for\_cloud\_backends:130`): `\_read\_json`
must pass a real `dst` (GCS/S3 `fetch` raises on `dst=None`); regression guard against silent
`{}`.
- ****Cold-start poll loop**** (`test\_cloud\_run\_polls\_until\_marker\_appears:153`): marker absent for
2 polls then present ? poll loops, doesn't resolve on poll 0.
- ****Worker wiring**** (`test\_cmd\_infer\_dispatches\_when\_cloud\_target:186`): `cmd\_infer` with
`compute\_target=cloud\_run` hands off to the backend (no in-process run).
`test\_worker\_writes\_done\_marker\_on\_success:209` / `test\_worker\_no\_done\_uri\_is\_unchanged:245`:
the container writes the done-marker only when `--done-uri` is passed (default-OFF otherwise).
- ****Dockerfile structural guard**** (`test\_dockerfile\_has\_required\_directives:277`): no build,
just asserts the load-bearing directives exist (see §3).

****Fakes/patterns available to reuse:****

- **`\_FakeRunClient`** (`:21`) ? subclasses `CloudRunJobClient`; on dispatch synchronously writes
`outputs/<vid>/report.json` + the done-marker into a local tmp "bucket" so the poll resolves
immediately. Ctor knobs: `video\_id`, `segment\_count`, `returncode`. Records every dispatch in
`self.calls`.
- **`\_NoopRunClient`** (`:45`) ? records the dispatch but writes nothing (for tests that monkeypatch
the storage layer directly, e.g. a `gs://` out_uri with no local dir).
- **`\_FAST`** `ExecutionPolicy(poll\_every\_n\_sec=0.0, max\_wait\_sec=5.0)` (`:18`) ? no-wait policy.
- **`\_cfg(compute\_target, storage\_backend)`** (`:59`) ? `AppConfig.model\_validate` helper.
- Storage monkeypatch pattern: `monkeypatch.setattr(cr, "fetch", ...)` , `monkeypatch.setattr(cr,
"get_storage", ...)` with a `_FakeStore.exists()` (`:161`).
- Worker-level fakes: stub validator (`\_OK` with `model\_dump`), `monkeypatch` of
`validator\_registry.run\_all\_with\_context`, `emit\_indexer\_report`, and
`trajectory\_hybrid.get\_video\_duration`; uses `detector\_impl=stub` + `skip\_ai\_handoff` for a
deterministic GPU-free run.

The real client `GoogleCloudRunJobClient` (`cloud\_run.py:60`) lazy-imports `google-cloud-run`
and is `# pragma: no cover` ? never exercised in CI (owner-verified on the M7 run only).

2. deploy/cloudrun/Dockerfile + README

```
**Dockerfile** (`deploy/cloudrun/Dockerfile`, 62 lines):  
- **Base:** `FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04` (':18') ? GPU/CUDA image, CI does NOT build it.  
- **Image contents:** ffmpeg + `fonts-dejavu-core` (watermark) + git/curl/build-essential/python3 (':27-30'); a separate conda `wasb` env (py3.8 + `torch==1.11.0+cu113` / `torchvision==0.12.0+cu113`, ':39-40') cloned from `https://github.com/nttcom/WASB-SBDT.git` (':38') for the shuttle detector as a subprocess; the main app env is `pip install .` of this repo (':47-50').  
- **Entrypoint:** `ENTRYPOINT ["python3", "-m", "backend.worker"]` (':61') ? args (`infer --video-uri ? --out-uri ? --done-uri ? --segmenter wasb_hybrid --set ?`) are supplied per-execution by `CloudRunCompute`. So **yes, it runs `worker infer`** (the `infer` subcommand is injected by the dispatcher, not hard-coded as CMD).  
- **Weights/env needed:** `WASB_PYTHON=/opt/conda/envs/wasb/bin/python`, `WASB_REPO_DIR=/opt/models/WASB-SBDT`, `WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar`, `RALLY_DETECTOR_IMPL=native` (':54-57').  
**Weights are NOT baked in** (WASB-C3 ? academic provenance not cleared for commercial sharing, ':14-16'); staged at runtime via bucket fetch/mount, env points `$WASB_WEIGHTS` at them.
```

****The P3a micromamba/Miniconda gap**** ? the Dockerfile uses ****Miniconda****, not micromamba:
...

```
33 RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o /tmp/mc.sh \  
34     && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \  
35     && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main || true \  
36     && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r || true \  
37     && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \  
...
```

The two `conda tos accept ? || true` lines (':35-36') swallow failures (the `|| true`), and they pull the Anaconda defaults channels ? exactly the Anaconda-ToS / channel friction that motivates a micromamba swap. This installs the full `Miniconda3-latest` distribution from `repo.anaconda.com` (defaults channels, subject to Anaconda's commercial ToS), not a conda-forge-only micromamba bootstrap.

```
**README** (`deploy/cloudrun/README.md`): build via `gcloud builds submit --tag ? -f deploy/cloudrun/Dockerfile .` ($1); create the GPU Job via `gcloud run jobs create rally-worker --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi --max-retries 0 --task-timeout 3600` ($2); wire the dispatcher with `DEPLOYMENT_PROFILE=cloud-serving` + `CLOUD_RUN_JOB`/`CLOUD_RUN_REGION`/`GOOGLE_CLOUD_PROJECT` env ($3). Status: dispatch shim + image are **landed + mock-tested (M6)**; the real build+push+run on an L4 is the **M7 owner step**. Notes a known gotcha: a **failed** Job writes **no** marker ? dispatcher's bounded poll times out ? clean **rc 4** (a failure-marker is a follow-up).
```

3. "Ingest a cloud result back into the API DB" ? ABSENT (the item-1 gap)

****No such code exists.**** The cloud path stops at returning a `ComputeResult` (returncode + `outputs\_uri` + `video\_id` + `report`); nothing reads those bucket artifacts (`outputs/<vid>/intervals.csv`, `report.json`) back into the API's SQLite DB.

```
- `backend/worker/__main__.py::_dispatch_remote` (':118-139') calls `compute.run_infer(spec)` and **just returns `result.returncode`** ? it logs `video_id`/`outputs_uri` but never persists segments to a DB. (The non-dispatch inline path, by contrast, does write to `Database` at ':188'.)  
- The **API job worker** `backend/api/job_worker.py` is fully **in-process and cloud-unaware**: `_run_claimed_job` (':72') calls `_main._execute_processing(req, ?)` and `db_instance.mark_job_done(job_id, result)` ? it has **no** import/reference to `get_compute_backend`, `CloudRunCompute`, `run_infer`, `ComputeResult`, or `outputs_uri`. (`grep` of `backend/api` + `main.py` for those terms returns only the billing `compute_backend` string at `job_worker.py:124` and a capabilities-probe read of `compute_target` at `main.py:369`.)  
- `backend/main.py::_execute_processing` (':439') is the in-process `hash?validate?segment?report` pipeline; it `compute_video_id` locally and writes to
```

`db\_instance` directly ? it never dispatches to a compute backend nor consumes a remote
`outputs\_uri`.

So the only consumer of `compute\_target=cloud\_run` is the **CLI** `backend.worker infer` path
(`cmd\_infer:168`). The hosted **API** (`/api/process` ? job worker ? `\_execute\_processing`) has
zero cloud-dispatch wiring, and there is **no** "read the bucket result back into the API
DB" code anywhere ? that ingest is entirely net-new (item-1 gap confirmed).

Key paths:

- `C:\Users\avidu\Projects\bhi-cloudrun\tests\test\_compute\_backend.py`
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\cloud\_run.py`,
`backend\compute\\_\_init\_\_.py`, `backend\compute\base.py`
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\\_\_main\_\_.py` (dispatch `:118-170`, done-
marker `:142-156`, inline DB write `:188`)
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\api\job\_worker.py` (`\_run\_claimed\_job:72`,
`\_execute\_processing` call `:86`)
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\main.py::\_execute\_processing` (`:439`)
- `C:\Users\avidu\Projects\bhi-cloudrun\deploy\cloudrun\Dockerfile`, `deploy\cloudrun\README.md`

2. assistant (2026-06-22T14:46:29.557Z)

I'll verify the key code paths in the actual repo before finalizing the design.

3. user (2026-06-22T14:46:30.824Z)

df3b225 docs(nightly): one-stop operations manual for the nightly engine-regression (#325)

```
---  
__init__.py  
base.py  
cloud_run.py
```

4. assistant (2026-06-22T14:46:32.870Z)

Let me verify the key insertion points in the actual code.

5. user (2026-06-22T14:46:34.359Z)

```
439     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:  
440         """The full hash ? validate ? segment ? report pipeline for one video.  
441  
442         This is the former synchronous /api/process body, unchanged in behavior, now run on  
443         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict  
444         the sync endpoint used to return (UI compatibility); the per-video observability  
445         report is still emitted in `finally:` and grafted as response["reports"].  
446  
447         `progress` is an optional callable(stage: str) used to surface coarse job progress.  
448         Raises on unexpected internal errors ? the caller records those as a job failure  
449         (after this function has already restored the pre-run segments, see below).  
450         """  
451         notify = progress or (lambda stage: None)  
452  
453         notify("hashing")  
454         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged  
(identity ?  
455         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The  
original  
456         # req.video_path (the source URI) is kept for the DB record + report; only the file-  
reading  
457         # consumers (hash / validators / segmenter) use the resolved local path.  
458         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,  
"storage", None))  
459         video_id = compute_video_id(local_video)  
460         was_reingest = db_instance.get_video(video_id) is not None  
461
```

```

462     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
463     notify("validating")
464     logger.info(f"Running validations for {req.video_path}...")
465     val_results, val_context = registry.run_all_with_context(local_video, global_config)
466     failures = [r for r in val_results if not r.passed]
467
468     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
469     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
470     seg_name = req.segmenter or default_seg
471     segmenter_actual = None
472     segmentation_ran = False
473     response: Optional[Dict[str, Any]] = None
474     try:
475         if failures:
476             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
477             # status; it no longer destroys the video's prior good segments.
478             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
479             response = {
480                 "video_id": video_id,
481                 "status": "failed",
482                 "message": "Video failed validation checks.",
483                 "results": [r.model_dump() for r in val_results],
484             }
485             return response
486
487     # 2. Run segmenter & indexer
488     logger.info("[API] Video passed checks. Segmenting and indexing...")
489     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
490
491     segmenter_cls = segmenter_registry.get(seg_name)
492     if not segmenter_cls:
493         # Submit-time validation already 400s unknown names; this is the defensive
494         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
495         available = list(segmenter_registry.list_available().keys())
496         response = {
497             "video_id": video_id,
498             "status": "failed",
499             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
500             "results": [r.model_dump() for r in val_results],
501             "play_segments": [],
502         }
503         return response
504
505     logger.info(f"[API] Using segmenter: {seg_name}")
506     notify(f"segmenting ({seg_name})")
507     segmenter_actual = seg_name
508     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
509     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
510     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
511     segmenter = segmenter_cls(db_instance, global_config)
512     try:
513         result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
514     except Exception:
515         # A raising segmenter must not leave half-written rows: restore the pre-run
516         # state (same destroy-after-confirm contract as the Failure branch), then
let

```

```

517         # the job worker record the failure.
518         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
519         raise
520     segmentation_ran = True
521
522     if isinstance(result, Failure):
523         logger.error(f"[API] Segmenter failed: {result.message}")
524         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
525         response = {
526             "video_id": video_id,
527             "status": "failed",
528             "message": f"Segmenter '{seg_name}' failed: {result.message}",
529             "results": [r.model_dump() for r in val_results],
530             "play_segments": [],
531         }
532         return response
533
534     segments = result.value if hasattr(result, "value") else result
535     notify("finalizing")
536     _finalize_reingest(video_id, segments, play_wm, cand_wm)
537
538     response = {
539         "video_id": video_id,
540         "status": "success",
541         "message": f"Successfully indexed {len(segments)} play segments using
542 '{seg_name}' segmenter.",
543         "results": [r.model_dump() for r in val_results],
544         "play_segments": segments,
545     }
546     return response
547 finally:
548     # Always emit the per-video observability report (next to the video, or to
549     # report.output_dir). Never raises. Surface the paths in the response.
550     paths = emit_indexer_report(
551         db=db_instance, video_id=video_id, video_path=req.video_path,
552         config=global_config,
553         val_results=val_results, val_context=val_context,
554         segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
555         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
556     )
557     if paths and isinstance(response, dict):
558         response["reports"] = paths
559
560 @app.post("/api/process", status_code=202)
561 def process_video(req: ProcessRequest, request: Request):
562     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
563
564     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
565     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
566     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
567     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
568     """
569     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
570     single-user,

```

6. user (2026-06-22T14:46:34.641Z)

```

1     """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
2     P3).
3
4     Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
5     The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
6     that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
    re-claimed when due ? with no central scheduler (the DB is the clock).

```

```

7
8     The monkeypatch contract these functions rely on (and that tests depend on):
9     every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10    `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11    `backend.main` module object at call-time (`import backend.main as _main`), never as a
12    bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13    ...)`` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15    inside the functions also avoids the circular import at module load (main.py imports
this
16    module; this module must not import main at import time).
17    """
18    import logging
19    import threading
20    import traceback
21    from concurrent.futures import ThreadPoolExecutor
22    from typing import Any, Dict, Optional
23
24    logger = logging.getLogger(__name__)
25
26    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
27    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
28    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
29    # already parallelize their AI handoff internally via their own pools.
30    _job_executor: Optional[ThreadPoolExecutor] = None
31
32
33    def _get_executor() -> ThreadPoolExecutor:
34        """Lazy module-global executor; tests monkeypatch this for inline execution."""
35        global _job_executor
36        if _job_executor is None:
37            _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
38        return _job_executor
39
40
41    class JobWaiting(Exception):
42        """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
43        WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
44        NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
45        releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
46        is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
47
48        The wiring is additive: the production segmenter path never raises this today (zero
49        behaviour change), so a job runs exactly as before. The hook is what a later slice
50        (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
51        """
52
53        def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
54            super().__init__(f"job parked for {delay_sec:.0f}s")
55            self.delay_sec = max(0.0, float(delay_sec))
56            self.progress = progress
57
58
59    def _job_to_request(job: Dict[str, Any]):
60        """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
61        re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
62        the original submit. The row stores exactly the ProcessRequest fields."""
63        import backend.main as _main
64        return _main.ProcessRequest(
65            video_path=job["video_path"],

```



```

66         player_pool=job.get("player_pool"),
67         max_frames=job.get("max_frames"),
68         segmenter=job.get("segmenter"),
69     )
70
71
72 def _run_claimed_job(job: Dict[str, Any]) -> None:
73     """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
74     terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
75
76     Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
77     the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
78     result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
79     job self-scheduled and will be re-claimed when `next_poll_at` is due.
80     """
81     import backend.main as _main
82     db_instance = _main.db_instance
83     job_id = job["id"]
84     req = _main._job_to_request(job)
85     try:
86         result = _main._execute_processing(
87             req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
88         if result.get("video_id"):
89             db_instance.set_job_video_id(job_id, result["video_id"])
90             db_instance.mark_job_done(job_id, result)
91             _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF)
92     except _main.JobWaiting as w:
93         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
94         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
95         db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
96     except Exception as e:
97         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
98         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
99
100
101 def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) -> None:
102     """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
103     ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
104     (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
105     persist it.
106
107     ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
108     ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
109     FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
110     + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
111     With the placeholder pricebook the cost is 0 regardless.
112
113     Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
114     claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
115     import backend.main as _main
116
117     billing_cfg = getattr(_main.global_config, "billing", None)
118     if not billing_cfg or not getattr(billing_cfg, "enabled", False):
119         return
120     try:
121         usage = (result or {}).get("usage") or {}

```

```

122         _main.db_instance.record_job_cost(
123             job["id"], usage=usage, pricebook_version=billing_cfg.pricebook_version,
124             gpu_type=usage.get("gpu_type"), compute_backend=(result or
125             {}).get("compute_backend"),
126             owner_id=job.get("owner_id"), margin=billing_cfg.margin,
127             )
128         logger.info("Recorded cost for job %s (pricebook=%s).", job["id"],
129         billing_cfg.pricebook_version)
130     except Exception as e: # never wedge a completed job on bookkeeping
131         logger.warning("Cost recording for job %s failed (non-fatal): %s",
132         job.get("id"), e)
133
134 def _drain_due_jobs() -> int:
135     """One worker tick: claim and run every currently-runnable job (queued, or waiting
136     whose
137     `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
138     Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
139     picked
140     up by a FUTURE drain once due; we schedule that follow-up drain via
141     `_schedule_drain` so
142     a single-worker box keeps making progress with no central timer (the DB is the
143     clock).
144     Returns the number of jobs that reached a terminal/parked state this tick.
145     """
146     import backend.main as _main
147     ran = 0
148     while True:
149         job = _main.db_instance.claim_due_job()
150         if job is None:
151             break
152         ran += 1
153         _main._run_claimed_job(job)
154     # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
155     new
156     # submit. On the single-box executor this is a delayed re-drain; multi-worker just
157     relies
158     # on the next submit/drain. Best-effort ? never raises into the worker.
159     _main._schedule_next_drain_if_waiting()
160     return ran
161
162 def _schedule_next_drain_if_waiting() -> None:
163     """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
164     due, so a
165     self-scheduled job resumes with no further client submit. The drain itself re-checks
166     due-ness, so an early wake is harmless and a clamped wake for a far-future job is
167     fine."""
168     import backend.main as _main
169     try:
170         delay = _main.db_instance.seconds_until_next_due()
171     except Exception as e: # pragma: no cover - defensive; never wedge the worker
172         logger.error(f"Could not compute next-due delay: {e}")
173         return
174     if delay is None:
175         return # nothing waiting
176     # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
177     ness.
178     _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))
179
180 def _arm_wake_timer(delay_sec: float) -> None:
181     """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
182     scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests

```

```

can
175         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting. """
176         import backend.main as _main
177         timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
178         timer.daemon = True
179         timer.start()
180
181
182     #: A parked job further out than this is re-checked by a capped wake rather than one
long
183     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
184     _MAX_DRAIN_WAKE_SEC = 60.0
185

```

7. assistant (2026-06-22T14:46:38.187Z)

Now let me look at how segments get persisted (`add\_play\_segment`) and the `\_finalize\_reingest` helper, plus the intervals.csv writer signature, to design the ingest precisely.

8. user (2026-06-22T14:46:39.147Z)

```

283     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
284         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
285
286         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
287         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
288         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
289         """
290         if segments:
291             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
292         else:
293             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
294
295
296     # --- API Endpoints ---
297     @app.get("/api/config")
298     def get_config():
299         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
300         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
301         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
302         # by tests/test_redact.py + the /api/config redaction test).
303         if global_config is None:
304             return {}
305         return redact_secrets(global_config.model_dump(mode="json"))
306
307     @app.get("/api/health")
308     def health():
309         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
310         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
311         confirm the engine is reachable and which detector is wired. Never raises."""
312         sport = getattr(global_config, "sport", None)

```

9. user (2026-06-22T14:46:40.570Z)

```

405     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
406         """Register or update a video row WITHOUT touching its child segments.

```

```

407
408         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
409         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
410         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
411         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
412         row in place; clearing segments is now an explicit, deliberate operation
413         (clear_video_data / prune_segments).
414         """
415         return self._execute_write(
416             "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
417             "ON CONFLICT(id) DO UPDATE SET "
418             "filepath=excluded.filepath, status=excluded.status, "
419             "validation_results=excluded.validation_results",
420             (video_id, filepath, status, json.dumps(validation_results)),
421             "Failed to add video",
422         )
423
424     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
425         """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
426
427         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
428         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
429         """
430         with self._connect() as conn:
431             cur = conn.cursor()
432             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
433                             (video_id,)).fetchone()[0]
434             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
435                             (video_id,)).fetchone()[0]
436             return int(p), int(c)
437
438     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
439                       play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
440                       cand_after: Optional[int] = None) -> None:
441         """Delete play/candidate segments by id range (events cascade from
play_segments).
442
443         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
444         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
445         and keep the OLD ones when the run failed/produced nothing (`*_after`).
446         """
447         with self._connect() as conn:
448             cur = conn.cursor()
449             if play_upto is not None:
450                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
451             if play_after is not None:
452                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
453             if cand_upto is not None:
454                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
455             if cand_after is not None:
456                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
457             conn.commit()

```

```

458
459     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
460         try:
461             with self._connect() as conn:
462                 cursor = conn.cursor()
463                 cursor.execute(
464                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
465                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
466                 )
467                 conn.commit()
468                 return cursor.lastrowid
469         except Exception as e:
470             logger.error(f"Failed to add play segment: {e}")
471             return None
472
473     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
474         return self._execute_write(
475             "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
476             (segment_id, timestamp, event_type, player, json.dumps(details or {})),
477             "Failed to add event",
478         )
479

```

10. user (2026-06-22T14:46:40.969Z)

```

73     def _write_intervals_csv(segments: List[dict], path: str) -> None:
74         """Decimal-second `[start,end)` rallies + shot count / ending reason ? the join-
friendly
75         artifact (same schema as the rally-annotator labels)."""
76         with open(path, "w", newline="") as f:
77             w = csv.writer(f)
78             w.writerow(["start", "end", "shot_count", "ending_reason"])
79             for s in segments:
80                 w.writerow([
81                     f'{float(s.get("start_time", 0.0)):.3f}',
82                     f'{float(s.get("end_time", 0.0)):.3f}',
83                     s.get("shot_count", ""),
84                     s.get("ending_reason", s.get("shot_count_confidence", "")),
85                 ])
86
87
88     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
89         with open(path, "w") as f:
90             json.dump({
91                 "video_id": video_id,
92                 "status": status,
93                 "segment_count": len(segments),
94                 "segments": [{"start": float(s.get("start_time", 0.0)),
95                             "end": float(s.get("end_time", 0.0))} for s in segments],
96             }, f, indent=2)
97

```

11. assistant (2026-06-22T14:46:43.986Z)

Now I have a complete picture. Let me check the `play\_segments` shape returned in the success response (what `segments` looks like) so the ingest produces an identical dict, and confirm the storage read helpers available.

12. user (2026-06-22T14:46:45.877Z)

```
564     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
565         """
566         Dynamically queries play segments based on filters:
567         - duration_min: float
568         - server: string
569         - receiver: string
570         - event_type: string (e.g. smash, foul)
571         """
572         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
573         params = [video_id]
574
575         if filters.get("duration_min"):
576             query += " AND r.duration >= ?"
577             params.append(filters["duration_min"])
578
579         if filters.get("server"):
580             query += " AND r.server = ?"
581             params.append(filters["server"])
582
583         if filters.get("receiver"):
584             query += " AND r.receiver = ?"
585             params.append(filters["receiver"])
586
587         if filters.get("event_type"):
588             # Check if there is an event of a specific type inside this segment
589             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
590             params.append(filters["event_type"])
591
592         segments_list = []
593         with self._connect() as conn:
594             cursor = conn.cursor()
595             rows = cursor.execute(query, params).fetchall()
596
597             for row in rows:
598                 r_dict = dict(row)
599                 r_id = r_dict["id"]
600                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
601
602                 # Fetch associated events
603                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
604                 r_dict["events"] = []
605                 for er in event_rows:
606                     e_dict = dict(er)
607                     e_dict["details"] = json.loads(e_dict["details"] or "{}")
608                     r_dict["events"].append(e_dict)
609
610                 segments_list.append(r_dict)
611
612         return segments_list
613
```

13. user (2026-06-22T14:46:46.293Z)

```
82     class CloudRunCompute(ComputeBackend):
83         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85         def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86             project: Optional[str] = None, storage_config: Optional[dict] = None,
87             policy: ExecutionPolicy = DEFAULT_POLICY,
88             token: Optional[str] = None,
```

```

89         container_overrides: Optional[tuple[str, ...]] = None) -> None:
90     self._client = run_client
91     self._job_name = job_name
92     self._region = region
93     self._project = project
94     self._storage_config = storage_config or {}
95     self._policy = policy
96     # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97     # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98     self._token = token
99     self._container_overrides = (
100         _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101     )
102
103     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104         out_root = spec.out_uri.rstrip("/")
105         token = self._token or uuid.uuid4().hex
106         done_uri = f"{out_root}/_dispatch/{token}.done"
107         argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108             "--done-uri", done_uri]
109         if spec.segmenter:
110             argv += ["--segmenter", spec.segmenter]
111         for kv in (*spec.config_overrides, *self._container_overrides):
112             argv += ["--set", kv]
113
114         execution = self._client.run_job(self._job_name, argv, region=self._region,
project=self._project)
115         LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
self._job_name, execution, done_uri)
116
117         marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
label="cloud-run-infer", logger=LOG)
118         video_id = str(marker.get("video_id", ""))
119         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
120         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
121         rc = int(marker.get("returncode", 1))
122         if not video_id:
123             LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
124                 "video_id) ? treating as failure", self._job_name)
125             rc = rc or 1
126         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
127         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
128         LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
129         return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
130
131     # --- storage-backed completion signal (bucket-poll) ---
132     def _read_marker(self, done_uri: str) -> Optional[dict]:
133         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
134         ref = parse_uri(done_uri)
135         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
136             return None
137         return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)
138
139     def _read_json(self, uri: str) -> dict:

```

```

143         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
144         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146         try:
147             ref = parse_uri(uri)
148             dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149             local = fetch(uri, dst, config=self._storage_config)
150             with open(local, "r", encoding="utf-8") as f:
151                 data: Any = json.load(f)
152             return data if isinstance(data, dict) else {}
153         except Exception:
154             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155             return {}
156

```

14. assistant (2026-06-22T14:46:50.779Z)

I have everything I need. Let me check how `out\_uri`/`video\_uri` would be derived for the API path (does ProcessRequest carry a gs:// URI, and what config field holds the out bucket).

15. user (2026-06-22T14:46:53.229Z)

```

324:     # input name is resolved against `input_root` (e.g.
`input_root="gs://bucket/videos"` + ".mp4"
325:     # => `gs://bucket/videos/x.mp4`) or, if `input_root` is empty, prefixed with
`input_backend`
327:     # (`gs://?`/`s3://?`) or an absolute local path ALWAYS wins. Non-local inputs are
fetched to a
340:     # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
365:     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""

```

16. user (2026-06-22T14:46:53.532Z)

```

259     class ProcessRequest(BaseModel):
260         video_path: str
261         player_pool: Optional[List[str]] = None
262         max_frames: Optional[int] = None
263         segmenter: Optional[str] = None
264         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
265         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
266         locale: Optional[str] = None
267
268     class QueryRequest(BaseModel):
269         video_id: str
270         server: Optional[str] = None
271         receiver: Optional[str] = None
272         duration_min: Optional[float] = None
273         event_type: Optional[str] = None

```

17. user (2026-06-22T14:46:56.699Z)

```

315         Google Drive (`gdrive.mount_root`). Per-backend settings are loose dicts passed to
the constructor ?
316         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
317         (boto3/google auth chains supply credentials). See `backend/storage/`.
318         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"

```



```

319         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
320         output_root: str = ""
321         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
322         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
325         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
327         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
330         input_root: str = ""
331         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
332         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
333         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
334         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
335         # the legacy flat layout is used unchanged.
336         single_folder_per_video: bool = False
337         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338         workspace_root: str = ""
339         # Cloud namespace prefix under the root so per-video folders sit tidily beside
340         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>). "" = root
directly.
341         workspace_prefix: str = "workspaces"
342         local: dict[str, Any] = Field(default_factory=dict)
343         s3: dict[str, Any] = Field(default_factory=dict)
344         gcs: dict[str, Any] = Field(default_factory=dict)
345         gdrive: dict[str, Any] = Field(default_factory=dict)
346
347
348     class DeploymentConfig(BaseModel):
349         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
350         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
351
352         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
353         behaves exactly as before this section existed. A profile is opt-in (``config.json``
354         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
355         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
356         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
357         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
358
359         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
360
361         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
362         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
363         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
364         # (recorded for observability; consumed by the ComputeBackend seam from M2+).

```

```
365         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
366
367
368     class BillingConfig(BaseModel):
369         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
```

18. assistant (2026-06-22T14:46:59.826Z)

I now have a complete, verified picture. The ``storage.input_root`/`output_root`` and ``workspace`` fields give me the bucket roots for deriving ``video_uri`/`out_uri``. I have all I need to produce the design.

19. user (2026-06-22T14:48:05.701Z)

Structured output provided successfully

20. assistant (2026-06-22T14:48:13.430Z)

Design complete. I verified every load-bearing claim against the actual ``bhi-cloudrun`` repo (HEAD ``df3b225``) and returned the full structured design via `StructuredOutput`.

Key findings: the compute seam (``backend/compute/``) and its offline-fake test harness (``tests/test_compute_backend.py`` with ``_FakeRunClient``) already exist but are consumed ONLY by the worker CLI. The API path (``/api/process`` ? ``job_worker._run_claimed_job`` ? ``main._execute_processing:439``) is entirely in-process and cloud-unaware ? the cloud-dispatch + result-ingest is net-new. The minimal diff adds one gated branch in ``_execute_processing`` (just before the local segmenter build at ``main.py:511``) that dispatches via ``get_compute_backend(...).run_infer(spec)`` and ingests ``intervals.csv`` from the bucket into ``play_segments`` through the same ``add_play_segment`` + ``_finalize_reingest`` lifecycle, so ``/api/query`` and ``/api/compile`` keep working unchanged and the path stays byte-identical unless ``compute_target=="cloud_run``.